

Multi-Model Ensemble with Entity-Level Disambiguation for Vision-Language Food Matching

Lei Jiang (Microsoft and flavored.ai)
for MetaFood VLM Challenge (at CVPR 2026)

Abstract

We present a multi-stage pipeline for matching food images to text captions across two distinct tasks: multi-label ingredient recognition (Task 1) and one-to-one dense caption assignment (Task 2). Our approach combines a four-model contrastive embedding ensemble with adaptive thresholding, Sinkhorn optimal transport, entity-level matching from open-source VLM descriptions, and targeted disambiguation using VLMs as binary classifiers. The system achieves a public leaderboard score of **0.69270** for Phase 1. All inference components use open-source models only, with no closed-source or frontier model outputs in the submission pipeline.

1 Introduction

The Dishcovery Mission II challenge ¹ requires participants to match food images to textual descriptions across two test sets:

- **Task 1 (Multi-label):** For each of 6,756 images, predict a subset of 399 short ingredient captions (e.g., “chicken”, “rice”, “basil”). Evaluation metric: F1 score.
- **Task 2 (Single-label, 1-to-1):** For each of 2,892 images, assign exactly one of 4,650 dense captions (e.g., “*Crispy golden fried chicken pieces served on a rustic wooden board*”), with 1,758 decoy captions. Each caption can be assigned to at most one image. Evaluation metric: accuracy.

The final score is the harmonic mean of Task 1 F1 and Task 2 accuracy. Competition rules prohibit using closed-source or pay-to-use models for generating any aspect of the final submission.

Our solution addresses the core challenges of each task: variable label density in Task 1 (images range from 1 to 17 ingredients), and semantic near-duplicates in Task 2 (captions differing only in one key ingredient but otherwise identical in phrasing). The pipeline evolves from embedding-only baselines through a series of principled optimizations, each validated on the Kaggle public leaderboard.

2 Multi-Model Embedding Ensemble

2.1 Model Selection

We ensemble four contrastive/embedding models spanning two distinct architecture families to maximize prediction diversity:

The key diversity axis is architectural: SigLIP2 [Tschannen et al., 2025] and CLIP [Radford et al., 2021] are *contrastive-only* encoders (separate image and text towers trained to align embeddings

¹Challenge site: <https://www.kaggle.com/competitions/dishcovery-mission-ii-cvpr-2026>

Table 1: Models used in the four-model ensemble.

Model	Architecture	Dim	Training Data	Family
SigLIP2 ViT-B-16-256	ViT-B	768	WebLI	Contrastive (sigmoid)
Qwen3-VL-Embed-8B	Multimodal LLM	4096	Alibaba internal	LLM-based
CLIP ViT-H-14 (DFN5B)	ViT-H	1024	5B filtered pairs	Contrastive (softmax)
Qwen3-VL-Embed-2B	Multimodal LLM	2048	Alibaba internal	LLM-based

via sigmoid [Zhai et al., 2023] or softmax [van den Oord et al., 2018] loss), while the two Qwen models [Bai et al., 2025] are *LLM-based* (a full language model backbone repurposed as an embedding encoder). These two families produce systematically different error patterns—contrastive models excel at short, distinctive labels while LLM-based models better handle compositional and long-form text—making their ensemble more effective than combining models within a single family. All models are accessed through the open_clip [Cherti et al., 2023] and Hugging Face libraries. The CLIP ViT-H-14 checkpoint was trained on 5 billion data-filtered samples [Fang et al., 2024].

We evaluated additional models including GME-Qwen2-VL-7B [Zhang et al., 2025, Wang et al., 2024], MetaCLIP2 ViT-H-14 [Xu et al., 2025], Gemma4 [Gemma Team et al., 2024], and fine-tuned Qwen3-VL-2B variants. All were rejected: GME and MetaCLIP2 were too highly correlated with existing CLIP-family members to provide ensemble diversity, Gemma4 had near-zero standalone Task 1 accuracy, and fine-tuned models consistently regressed on Kaggle despite local improvements (Section 7).

2.2 Score Fusion

For each model m , we compute a raw similarity matrix $S_m = X_m Y_m^\top$ where X_m and Y_m are L2-normalized image and text embeddings respectively. We then apply per-row z-normalization:

$$z_m^{(i,j)} = \frac{S_m^{(i,j)} - \mu_i}{\sigma_i}, \quad (1)$$

where μ_i and σ_i are the mean and standard deviation of row i . This normalization is critical: it projects all models onto a common scale regardless of embedding dimensionality (768 vs. 4096) and score distribution.

The ensemble score is a weighted sum with task-specific weights:

$$\text{score}^{(i,j)} = \sum_m w_m \cdot z_m^{(i,j)}. \quad (2)$$

Task 1 weights: [0.25, 0.35, 0.30, 0.10] (SigLIP2, Qwen3-8B, CLIP-H, Qwen3-2B).

Task 2 weights: [0.20, 0.40, 0.15, 0.25].

The weight difference reflects each task’s nature. Task 1’s short ingredient labels benefit from classical contrastive models (SigLIP2 and CLIP-H receive 55% combined weight). Task 2’s dense descriptive captions favor the language-aware Qwen models (65% combined weight), with Qwen3-2B receiving substantially more weight (0.25 vs. 0.10) because its distinct capacity creates useful error diversity for long-form matching.

3 Task 1: Adaptive- k Selection

3.1 Motivation

Task 1 requires predicting a variable number of ingredient labels per image. The actual label density varies widely (mean ≈ 1.9 , range 1–17), making any fixed- k strategy suboptimal. Fixed top-5 selection produces an average of 5.0 predictions per image—far too many for most images, inflating false positives.

3.2 Gap-Based Selection

We observe that ensemble similarity scores form a natural cluster of “present” items followed by a sharp drop-off. The largest gap in the top scores separates truly present ingredients from background noise.

The algorithm examines the top $sr = 5$ scores, computes consecutive gaps, and selects $k = \arg \max(\text{gaps}) + 1$, capped at $\text{max_k} = 5$:

Algorithm 1 Adaptive gap-based k selection

Require: Score vector $\mathbf{r}$ of length C ; search range $sr = 5$; cap $\text{max_k} = 5$

```
1: ranked  $\leftarrow$  sort_descending( $\mathbf{r}$ )
2: gaps[ $i$ ]  $\leftarrow$  ranked[ $i$ ] – ranked[ $i+1$ ]   for  $i = 0, \dots, sr-1$ 
3:  $k \leftarrow \arg \max(\text{gaps}) + 1$ 
4: return max(1, min( $k$ , max_k))
```

Example. For an image of steak with rosemary, the ensemble produces the following top-5 z-scores:

Rank	Caption	Score	Gap
1	steak	4.81	0.89
2	rosemary	3.92	2.68 $\leftarrow$ largest
3	plate	1.24	0.31
4	basil	0.93	0.12
5	cream	0.81	—

The largest gap occurs after rank 2, so $k = 2$: predict {steak, rosemary}. A fixed top-5 would add three false positives.

This single change improved Task 1 F1 from 0.47 to 0.74 and reduced average predictions per image from 5.0 to ~ 1.9 , closely matching the ground-truth density. When reporting the task-specific F1 score, it’s calculated through a local estimation rather than what Kaggle provided.

3.3 Category-Specific Thresholds

Error analysis revealed two systematic categories of false negatives:

- **Seasonings** (9 ingredients, e.g., balsamic dressing, basil, rosemary, sesame seeds): visually subtle elements often overshadowed by the main dish. Z-score threshold: 3.3.
- **Containers** (5 ingredients, e.g., breading, wafer cone, tart crust, sweet bun): vessels or wrappers visually dominated by their contents. Z-score threshold: 3.2.

For these 14 ingredients, if the z-score exceeds the category threshold but the ingredient was not selected by adaptive- k , it is added to the prediction. This recovered seasonings and containers without affecting other ingredient categories.

4 Task 2: Sinkhorn Optimal Transport Assignment

4.1 Problem Structure

Task 2 is a bipartite matching problem: 2,892 images must be assigned to 2,892 of 4,650 captions (1,758 are decoys), with each caption used at most once. Naïve greedy argmax (each image picks its highest-scoring caption) resulted in 927 collisions out of 2,892 images.

4.2 From Hungarian to Sinkhorn

We initially applied the Hungarian algorithm [Kuhn, 1955] for exact optimal 1-to-1 assignment, which improved Kaggle score from 0.584 (greedy) to 0.646 (+0.062). However, the Hungarian algorithm is sensitive to noise in the cost matrix: a single mis-scored entry can cascade through the global assignment.

We replaced it with Sinkhorn optimal transport [Cuturi, 2013], which iteratively normalizes the exponentiated cost matrix to approximate a doubly-stochastic matrix:

$$K = \exp\left(\frac{\text{score}}{\tau}\right), \quad (3)$$

$$u^{(t+1)} = \frac{\mathbf{1}}{K v^{(t)}}, \quad v^{(t+1)} = \frac{\mathbf{1}}{K^\top u^{(t+1)}}, \quad (4)$$

with temperature $\tau = 0.15$ and 50 iterations. The resulting soft transport matrix $P = \text{diag}(u) K \text{diag}(v)$ is then converted to a hard assignment via confidence-ordered greedy: images are processed in descending order of $\max_j P_{ij}$, and each assigned caption is removed from the pool.

Sinkhorn’s advantage is *noise robustness*: by softening the cost matrix before assignment, it diffuses confidence across near-ties, which is particularly helpful for Task 2’s many near-duplicate paraphrased captions. This contributed to a Kaggle improvement from 0.656 (Hungarian) to 0.672 (Sinkhorn).

5 Entity-Level Matching via Description Model

5.1 The Embedding Similarity Failure Mode

Contrastive embeddings encode holistic visual and textual similarity—style, composition, color palette, and sentence structure. When two captions describe dishes that *look* similar but contain *different key ingredients*, the embedding cannot distinguish them.

Real example. Image `fs103_4750.jpg` shows grilled salmon with broccoli.

Both captions share broccoli and describe vibrant, healthy plating. The embedding scores them nearly identically. The critical difference—salmon vs. salad—is invisible to the embedding.

Table 2: Embedding failure: stylistically similar captions with different key ingredients.

	Caption	Key entities
Predicted	<i>“A vibrant salad featuring fresh broccoli, red onion, dried cranberries, and raisins, all mingled together in a creamy sauce...”</i>	broccoli, onion, cream sauce
Correct:	<i>“A plate featuring tender broccoli alongside a perfectly baked salmon fillet...”</i>	broccoli, fish

5.2 Description Cache Construction

We use **Qwen3-VL-8B-Instruct** (an open-source VLM) to generate structured food descriptions for every test image. The model receives each image with a carefully designed prompt requesting categorized food item listings (protein, starch, vegetable/fruit, sauce/condiment, garnish, beverage) using specific common names.

Descriptions are cached in a JSON file for reproducibility. The cache is resume-safe: images with existing descriptions are skipped on re-runs, enabling incremental generation. For the final pipeline, we also generate descriptions using **Gemma4** as a secondary signal for disambiguation.

5.3 Entity Extraction and Jaccard Bonus

From each image description and each candidate caption, we extract food entities using a vocabulary of 66 food terms with regex patterns. The entity Jaccard similarity [Jaccard, 1912] between an image’s description entities $E(d_i)$ and a caption’s entities $E(c_j)$ is computed:

$$J(d_i, c_j) = \frac{|E(d_i) \cap E(c_j)|}{|E(d_i) \cup E(c_j)|}. \quad (5)$$

This score is added as a bonus to the Sinkhorn cost matrix:

$$\text{final_score}_{ij} = \text{embedding_score}_{ij} + \lambda \cdot J(d_i, c_j), \quad (6)$$

with $\lambda = 0.20$. The bonus acts as a tiebreaker: it only flips the assignment when two captions are already close in embedding space, which is precisely the failure mode it targets.

In the salmon example above: the description yields entities {fish, broccoli, onion, salad}. The gold caption’s entity {fish} matches, while the predicted caption has no fish entity. The Jaccard bonus correctly steers the assignment.

Impact: +12 correct Task 2 assignments locally ($1,366 \rightarrow 1,378$), Kaggle $0.672 \rightarrow 0.674$.

5.4 Why Entity Extraction Outperformed Embedding of Descriptions

We initially attempted a describe-then-embed approach: vectorize the description text with Qwen3’s text encoder and compute cosine similarity to vectorized captions. This *regressed* on Kaggle (0.671 vs. 0.672 baseline).

The failure is instructive: embedding a full description recaptures the same holistic similarity problem—style words, adjectives, and scene descriptions dominate the vector, diluting the food entity signal. Extracting discrete entities and computing set overlap removes all stylistic noise and focuses exclusively on what matters: *which foods are present*.

6 Granular Optimizations

6.1 VLM-Based Binary Disambiguation (Task 1)

6.1.1 Triggered Yes/No Rules

For 30 high false-negative ingredients, we ran both Qwen3-VL-8B-Instruct and Gemma4 as binary classifiers, asking each image: *“Does this image contain [ingredient]? Answer yes or no.”* Results are cached per (model, ingredient, image) triple.

A disambiguation rule fires only when:

1. The ingredient appears in the embedding ensemble’s **top- K candidates** (it is “triggered”), **and**
2. At least one VLM confirms the ingredient is present.

This constraint prevents hallucination: the VLM can only add labels that the embedding already considers plausible.

Effect: Task 1 F1 improved from 0.777 $\rightarrow$ 0.788 (+0.011).

6.1.2 Margin Gates

The basic triggered rules used a fixed top-10 rank gate, which was too loose—candidates at rank 9–10 are often noise. Margin gates add per-ingredient scoring filters:

- **Rank gate:** The ingredient must be within the top- K (swept: $K \in \{5, 6, 7, 8, 10\}$).
- **Boundary gap gate:** The score gap between the adaptive- k cutoff and the ingredient’s score must be $\leq$ threshold. This ensures the ingredient “just missed” the adaptive- k selection.
- **Top gap gate:** The gap between the image’s top score and the ingredient’s score must be $\leq$ threshold, ensuring the ingredient is not too far from the dominant signal.

Each ingredient selects its own optimal gate configuration. Visually subtle ingredients (e.g., basil) use loose gates; visually obvious ingredients (e.g., chicken) use tight gates.

Effect: Task 1 F1 improved from 0.788 $\rightarrow$ 0.798 (+0.010).

6.2 Clustered A/B Corrections (Task 1 and Task 2)

6.2.1 Mechanism

Error analysis revealed systematic confusion between semantically similar ingredient pairs (e.g., chicken $\leftrightarrow$ pork, beef $\leftrightarrow$ steak, rice $\leftrightarrow$ noodles). Clustered A/B corrections address these by using the VLM descriptions as an oracle:

For each frequent confusion pair (A, B) :

1. Identify images where the embedding predicts A .
2. Check the Qwen3 description: if it mentions B but not A , apply a correction—remove A and add B (Task 1), or nudge the caption score (Task 2).

This is the only mechanism that improved both precision and recall simultaneously, because it fixes *systematic errors* rather than adjusting thresholds.

6.2.2 Progression

Corrections were applied in three passes with increasing aggressiveness:

Table 3: Clustered A/B correction progression on Task 1.

Pass	Scope	Task 1 F1
Conservative	High-frequency pairs, strict description gate	0.813
Aggressive	Extended pair set, relaxed gates	0.813+
Second pass	Residual corrections from re-analysis	0.829

Kaggle impact: The clustered A/B pipeline contributed the single largest Kaggle improvement of the entire project: +0.016 (0.674 $\rightarrow$ 0.690).

6.3 Rare-Entity Boost (Task 2)

Entities appearing in very few captions carry high discriminative power. For example, “lobster” appears in only 1 of 4,650 captions—if a VLM confirms an image contains lobster, the assignment is nearly deterministic.

For entities appearing in ≤ 5 captions, we query both Qwen3-8B and Gemma4. If both confirm the entity’s presence, the matching caption(s) receive a score boost of +0.075 in the Sinkhorn cost matrix before assignment. This is motivated by IDF weighting:

$$\text{IDF}(\text{“lobster”}) = \log \frac{4651}{2} = 7.75 \quad \text{vs.} \quad \text{IDF}(\text{“rice”}) = \log \frac{4651}{312} = 2.70.$$

Rare entities provide far more information per confirmation. This is analogous to the classical IDF weighting in information retrieval [Spärck Jones, 1972].

Effect: +9 correct Task 2 assignments locally (1,378 $\rightarrow$ 1,387).

6.4 Recall Recovery (Task 1)

After all disambiguation and correction passes, a final recall recovery step reviews ingredients that were not added by any previous mechanism. Using the same VLM caches, it identifies high-precision addition opportunities at a relaxed threshold ($p \geq 0.525$), recovering ingredients like french fries and ground meat that were systematically missed.

Effect: Task 1 F1 improved from 0.829 $\rightarrow$ 0.830, with a Kaggle gain of +0.003 (0.690 $\rightarrow$ 0.693).

7 Approaches Attempted and Rejected

We explored several directions that did not improve the final Kaggle score and were excluded from the production pipeline. *Fine-tuning* both SigLIP2 and Qwen3-VL-2B (on synthetic and web-image data) consistently regressed on Kaggle despite local gains, suggesting that the base models’ zero-shot diversity is more valuable than task-specific overfitting. *Adding a fifth ensemble model*—including GME-Qwen2-VL-7B [Zhang et al., 2025, Wang et al., 2024], MetaCLIP2 [Xu et al., 2025], and Gemma4 [Gemma Team et al., 2024]—provided no benefit because these models were too correlated with existing CLIP-family members to add diversity. *Describe-then-embed* for Task 2 (vectorizing full VLM descriptions instead of extracting entities) recaptured the same holistic similarity problem and regressed. *Semantic TF-IDF with Qwen3 description gates* showed +17 locally but dropped on Kaggle, revealing overfitting to the local Gemini judge. *Binary food classifiers* trained on web

images achieved only 67.6% accuracy—too weak for reliable vetoing. *Label synonym co-selection* and *post-Sinkhorn swap mechanisms* both hurt precision more than they helped recall. The key lesson across all rejected directions is that the local Gemini-based judge is a useful diagnostic tool but an unreliable optimization target: fine-grained tuning against it frequently overfits.

8 Score Progression

Table 4 summarizes the Kaggle public leaderboard score at each major milestone.

Table 4: Kaggle public leaderboard progression.

Date	Technique	Kaggle	Δ
Apr 18	SigLIP2 baseline	0.400	—
Apr 21	Qwen3-8B single model	0.438	+0.038
Apr 23	2-model ensemble + adaptive- k	0.577	+0.139
Apr 24	3-model ensemble	0.584	+0.007
Apr 25	+ Hungarian 1-to-1 assignment	0.646	+0.062
Apr 25	+ 4-model + split weights	0.656	+0.010
Apr 26	+ Seasoning boost	0.661	+0.005
Apr 27	+ Sinkhorn + seasoning Tier1 + container	0.672	+0.011
Apr 29	+ Entity Jaccard from descriptions	0.674	+0.002
May 1	+ VLM disambig. + clustered A/B	0.690	+0.016
May 1	+ Recall recovery	0.693	+0.003

9 Final Pipeline Summary

The production pipeline executes the following steps:

1. **Embedding caching:** Generate and cache image/text embeddings for all four models across both test sets.
2. **Description caching:** Generate Qwen3-VL-8B-Instruct and Gemma4 image descriptions for all test images.
3. **VLM yes/no caching:** For 30 probed ingredients (Task 1) and rare entities (Task 2), cache binary yes/no responses from both VLMs.
4. **Score fusion:** Compute z-normalized similarity matrices for each model, combine with task-specific weights (Eq. 2).
5. **Task 1 prediction:**
 - (a) Adaptive- k gap selection (Alg. 1)
 - (b) Seasoning and container category boosts
 - (c) Triggered disambiguation with margin gates
 - (d) Clustered A/B corrections
 - (e) Recall recovery
6. **Task 2 prediction:**
 - (a) Entity Jaccard bonus added to ensemble scores (Eq. 6)
 - (b) Rare-entity yes/no boost
 - (c) Clustered A/B score nudges
 - (d) Sinkhorn assignment ($\tau = 0.15$, 50 iterations)
 - (e) Confidence-ordered greedy finalization

All VLM-based components are cached and resume-safe. On new or private data, the pipeline regenerates missing caches automatically. If description or yes/no caches are unavailable (e.g., due to time constraints), the pipeline degrades gracefully to embedding-only inference. Total inference time on a single NVIDIA H100 GPU is within the one-hour competition limit.

10 Reproducibility

All code, model weights (accessible via HuggingFace repositories), and static rule manifests are included in the submission package. The pipeline requires Python 3.13, PyTorch with CUDA support, and Hugging Face access tokens for gated models (Qwen3-VL, Gemma4). No closed-source, proprietary, or frontier model outputs are used at any stage of inference. Gemini was used exclusively as a local diagnostic judge during development and does not contribute any features, labels, or rules to the final submission.

References

- Shuai Bai et al. Qwen3-VL technical report. *arXiv preprint arXiv:2511.21631*, 2025.
- Mehdi Cherti, Romain Beaumont, Ross Wightman, Mitchell Wortsman, Gabriel Ilharco, Cade Gordon, Christoph Schuhmann, Ludwig Schmidt, and Jenia Jitsev. Reproducible scaling laws for contrastive language-image learning. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.
- Marco Cuturi. Sinkhorn distances: Lightspeed computation of optimal transport. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 26, 2013.
- Alex Fang, Albin Madappally Jose, Amit Jain, Ludwig Schmidt, Alexander Toshev, and Vaishaal Shankar. Data filtering networks. In *International Conference on Learning Representations (ICLR)*, 2024.
- Gemma Team, Thomas Mesnard, Cassidy Hardin, Robert Dadashi, Surya Bhupatiraju, Shreya Pathak, Laurent Sifre, Morgane Rivière, Mihir Sanjay Kale, Juliette Love, Pouya Tafti, Léonard Hussenot, et al. Gemma: Open models based on gemini research and technology. *arXiv preprint arXiv:2403.08295*, 2024.
- Paul Jaccard. The distribution of the flora in the alpine zone. *New Phytologist*, 11(2):37–50, 1912.
- Harold W. Kuhn. The Hungarian method for the assignment problem. *Naval Research Logistics Quarterly*, 2(1–2):83–97, 1955.
- Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. Learning transferable visual models from natural language supervision. In *International Conference on Machine Learning (ICML)*, 2021.
- Karen Spärck Jones. A statistical interpretation of term specificity and its application in retrieval. *Journal of Documentation*, 28(1):11–21, 1972.
- Michael Tschannen, Alexey Gritsenko, Xiao Wang, Muhammad Ferjad Naeem, Ibrahim Alabdulmohsin, Nikhil Parthasarathy, Talfan Evans, Lucas Beyer, Ye Xia, Basil Mustafa, Olivier Hénaff,

- Jeremiah Harmsen, Andreas Steiner, and Xiaohua Zhai. SigLIP 2: Multilingual vision-language encoders with improved semantic understanding, localization, and dense features. *arXiv preprint arXiv:2502.14786*, 2025.
- Aäron van den Oord, Yazhe Li, and Oriol Vinyals. Representation learning with contrastive predictive coding. *arXiv preprint arXiv:1807.03748*, 2018.
- Peng Wang, Shuai Bai, Shengding Hu, Shijie Wang, Sinan Tan, Peng Yao, et al. Qwen2-VL: Enhancing vision-language model’s perception of the world at any resolution. *arXiv preprint arXiv:2409.12191*, 2024.
- Hu Xu, Zirui Wang, Fangyin Wei, Saining Xie, Shang-Wen Li, Ching-Feng Yeh, Shiyu Chang, Chunting Zhou, Faisal Azhar, Luke Zettlemoyer, Mike Lewis, and Puxin Xu. Meta CLIP 2: A worldwide scaling recipe for CLIP training. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.
- Xiaohua Zhai, Basil Mustafa, Alexander Kolesnikov, and Lucas Beyer. Sigmoid loss for language image pre-training. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- Xin Zhang, Yanzhao Zhang, Wen Xie, Mingxin Li, Ziqi Dai, Dingkun Long, Pengjun Xie, Meishan Zhang, Wenjie Li, and Min Zhang. GME: Improving universal multimodal retrieval by multimodal LLMs. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025.